

Encoder–Decoder Architecture

What it is

The Encoder–Decoder architecture is a neural network framework used for sequence-to-sequence (Seq2Seq) tasks, where an input sequence is converted into an output sequence. It is commonly used in applications such as machine translation, text summarization, speech recognition, and chatbots.

The architecture consists of two main parts:

1. **Encoder**
2. **Decoder**

The encoder processes the input sequence and converts it into a compact representation called a context vector, which the decoder uses to generate the output sequence.

Architecture of Encoder–Decoder

1. Encoder

The encoder reads the input sequence token by token and encodes it into a fixed-length vector representation.

Key points:

- Usually implemented using Recurrent Neural Networks (RNN), LSTM, or GRU.
- Captures the semantic meaning of the entire input sequence.
- Produces a hidden representation called the context vector.

Example:

Input sentence: “I love AI”

The encoder converts it into a numerical representation containing its meaning.

2. Context Vector

The context vector is the output of the encoder and contains the encoded information of the entire input sequence.

Key points:

- Acts as a bridge between the encoder and decoder.

- Represents the meaning of the input sentence.
- Passed as input to the decoder.

Limitation:

- For long sequences, a single context vector may not capture all information effectively.

3. Decoder

The decoder generates the output sequence one token at a time using the context vector and previously generated tokens.

Key points:

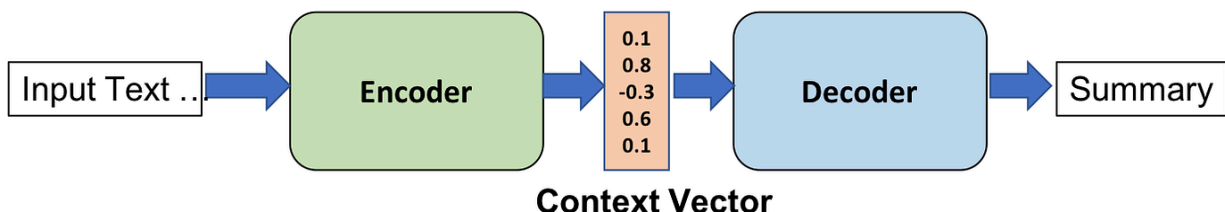
- Usually implemented using RNN, LSTM, or GRU.
- Predicts the next word based on the previous word and context vector.
- Continues until the end-of-sequence token is generated.

Example:

Input: "I love AI"

Output (French translation): "J'aime l'IA"

Encoder–Decoder Flow



Transformer Architecture

What it is

A Transformer is a deep learning architecture introduced in the paper “Attention Is All You Need” (2017). It is designed to process sequential data using self-attention mechanisms instead of recurrent networks. Unlike RNN-based models, transformers process all tokens in parallel, making them much faster and better at capturing long-range dependencies in text. Transformers are widely used in modern NLP models such as BERT, GPT, and T5.

Architecture of Transformer

The transformer architecture consists of two main components:

1. **Encoder Stack**
2. **Decoder Stack**

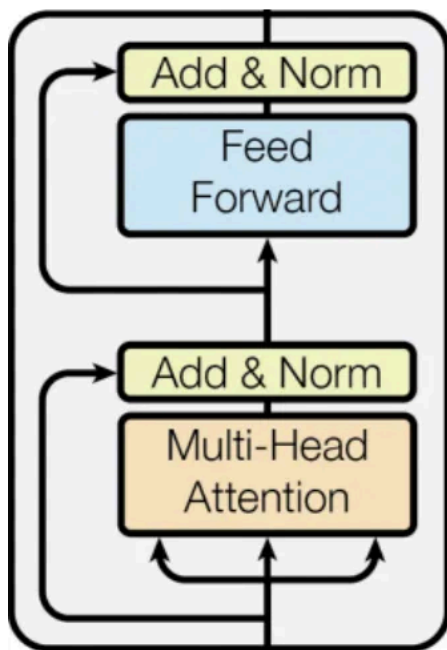
Input → Encoder Stack → Decoder Stack → Output

Transformer Encoder

Each encoder layer consists of the following components:

1. Input Embedding
2. Positional Encoding
3. Multi-Head Self-Attention
4. Feed Forward Neural Network
5. Add & Layer Normalization

Encoder Block Structure



Encoder layer

Multiple encoder layers are stacked together (typically 6 layers).

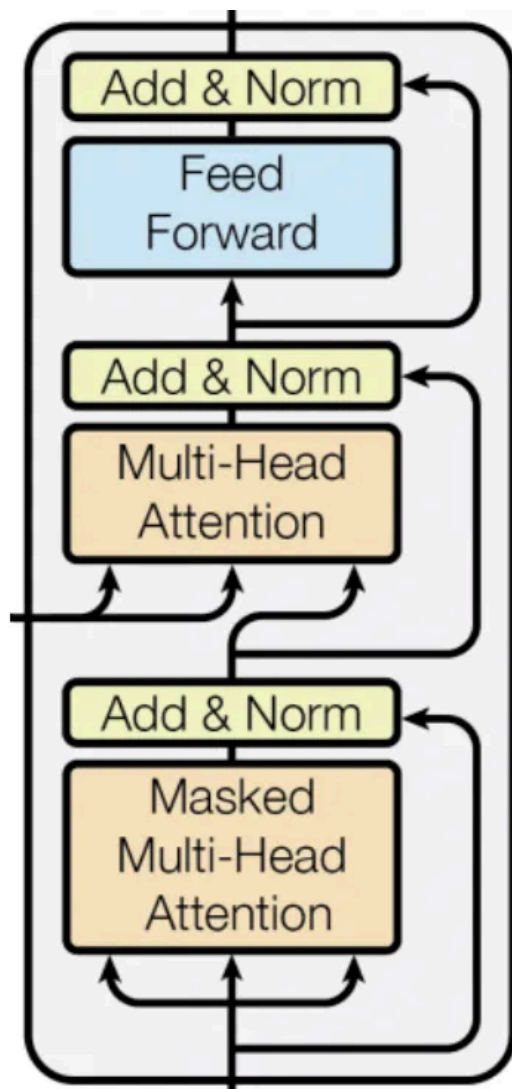
Transformer Decoder

The decoder is similar to the encoder but includes an additional encoder–decoder attention layer.

Each decoder layer contains:

1. Masked Multi-Head Self-Attention
2. Encoder–Decoder Attention
3. Feed Forward Neural Network
4. Add & Layer Normalization

Decoder Block Structure



Decoder layer

Transformer Workflow

1. Input tokens are converted into embeddings.
2. Positional encoding is added to preserve word order.
3. The encoder processes the sequence using self-attention and feed-forward layers.
4. The decoder generates the output sequence using masked attention and encoder information.
5. A linear layer and softmax produce the final predicted tokens.

Hands on practise in python